

DS — Programmation Orientée Objet (C#)

Sujet : Gestion de Datacenter et Virtualisation

Durée : 2h00 — Coefficient : 4

Contexte :

Vous développez le module central d'un orchestrateur de cloud privé. Votre mission est de modéliser les serveurs physiques (Hyperviseurs) et les serveurs virtuels (VMs) afin de gérer l'allocation des ressources (CPU, RAM, Disque).

Consignes Générales :

- **Aucun héritage ni polymorphisme n'est demandé.**
 - L'usage de la calculatrice est interdit.
 - Le code doit être écrit en C# valide.
 - Respectez strictement l'encapsulation (attributs privés, propriétés publiques).
 - Respectez les conventions de nommage (PascalCase, camelCase).
-

Partie 1 : La Machine Virtuelle (5 points)

Une machine virtuelle (VM) est un ordinateur simulé qui consomme des ressources.

Travail à faire :

Créez la classe `MachineVirtuelle` avec les éléments suivants :

1. **Propriétés (Lecture/Écriture ou Lecture seule selon logique) :**
 - `Id` (string) : Identifiant unique (ex: "VM-104").
 - `Nom` (string) : Nom de la machine (ex: "Web-Server-01").
 - `Vcpu` (int) : Nombre de processeurs virtuels.
 - `Ram` (int) : Quantité de mémoire vive en Go.
 - `Disque` (int) : Espace disque nécessaire en Go.
 - `EstActive` (bool) : État de la machine (Allumée/Éteinte).
2. **Constructeur :**
 - Permet d'initialiser l'`Id`, le `Nom`, le nombre de `Vcpu`, la `Ram` et le `Disque`.
 - Par défaut, une VM est créée éteinte (`EstActive` à `false`).
3. **Méthode `CalculerScorePerformance()` :**
 - Retourne un `double` .
 - Le score est calculé ainsi : $(Vcpu * 10) + (Ram * 5) + (Disque * 0.5)$.
 - Ce score représente la "puissance" théorique de la machine.

Partie 2 : L'Hyperviseur (8 points)

Un hyperviseur est un serveur physique puissant capable d'héberger plusieurs VMs.

Travail à faire :

Créez la classe `Hyperviseur` :

1. Propriétés & Attributs :

- `Nom` (string) : Nom du serveur physique (ex: "HV-Rack1-04").
- `MaxVcpu` (int), `MaxRam` (int), `MaxDisque` (int) : Capacités maximales du serveur.
- `_vms` : Une liste privée (`List<MachineVirtuelle>`) qui stocke les VMs hébergées sur cet hyperviseur.

2. Constructeur :

- Initialise le nom et les capacités maximales.
- Instancie la liste `_vms` (vide au départ).

3. Méthode `CalculerRessourcesUtilisees()` :

- Cette méthode ne retourne rien mais affiche dans la console (ou retourne une chaîne, au choix) le total des ressources consommées par les VMs **actives** uniquement.
- Exemple de sortie : "CPU: 12/32, RAM: 64/128 Go, Disque: 500/2000 Go".

4. Méthode `PeutAccueillir(MachineVirtuelle vm)` :

- Retourne `true` si l'hyperviseur a suffisamment de ressources libres (Vcpu, Ram ET Disque) pour accepter la nouvelle VM.
- *Attention : Vous devez additionner les ressources de TOUTES les VMs déjà présentes (qu'elles soient actives ou non, car l'espace est réservé) et vérifier si l'ajout de la nouvelle ne dépasse pas les Max.*

5. Méthode `AjouterVM(MachineVirtuelle vm)` :

- Utilise `PeutAccueillir` pour vérifier la faisabilité.
- Si c'est possible : ajoute la VM à la liste et affiche "VM ajoutée".
- Sinon : affiche "Erreur : Capacité insuffisante".

Partie 3 : Le Datacenter (7 points)

Le Datacenter regroupe plusieurs hyperviseurs et doit optimiser le placement des machines.

Travail à faire :

Créez la classe `Datacenter` :

1. Attributs :

- Une liste d'hyperviseurs `_parc`.

2. Méthode `RechercherHoteIdeal(MachineVirtuelle vm)` :

- Cette méthode doit trouver l'hyperviseur le plus adapté pour accueillir une nouvelle VM.
- **Algorithme de recherche :**
 - a. Parcourir tous les hyperviseurs du parc.
 - b. Identifier ceux qui peuvent accueillir la VM (via `PeutAccueillir`).
 - c. Parmi les candidats, sélectionner celui qui a le plus de RAM libre (pour privilégier la performance).
 - d. Retourner cet hyperviseur (ou `null` si aucun n'est disponible).

3. Méthode `AfficherStatistiquesGlobales()` :

- Affiche le nombre total d'hyperviseurs.
- Affiche le nombre total de VMs hébergées dans tout le datacenter.
- Affiche le taux d'occupation global du CPU (Somme des Vcpu utilisés / Somme des MaxVcpu * 100).

Annexe : Exemple d'utilisation (Main)

```
public static void Main(string[] args)
{
    // 1. Création du Datacenter
    Datacenter monCloud = new Datacenter();

    // 2. Création d'un Hyperviseur
    Hyperviseur hv1 = new Hyperviseur("HV-01", 32, 128, 2000); // 32 cœurs, 128Go RAM, 2To Disque
    monCloud.AjouterHyperviseur(hv1); // Supposons que cette méthode existe

    // 3. Création d'une VM
    MachineVirtuelle webVm = new MachineVirtuelle("VM-WEB", "Apache", 4, 8, 100);

    // 4. Recherche et placement
    Hyperviseur hote = monCloud.RechercherHoteIdeal(webVm);

    if (hote != null)
    {
        hote.AjouterVM(webVm);
        webVm.EstActive = true;
    }
}
```